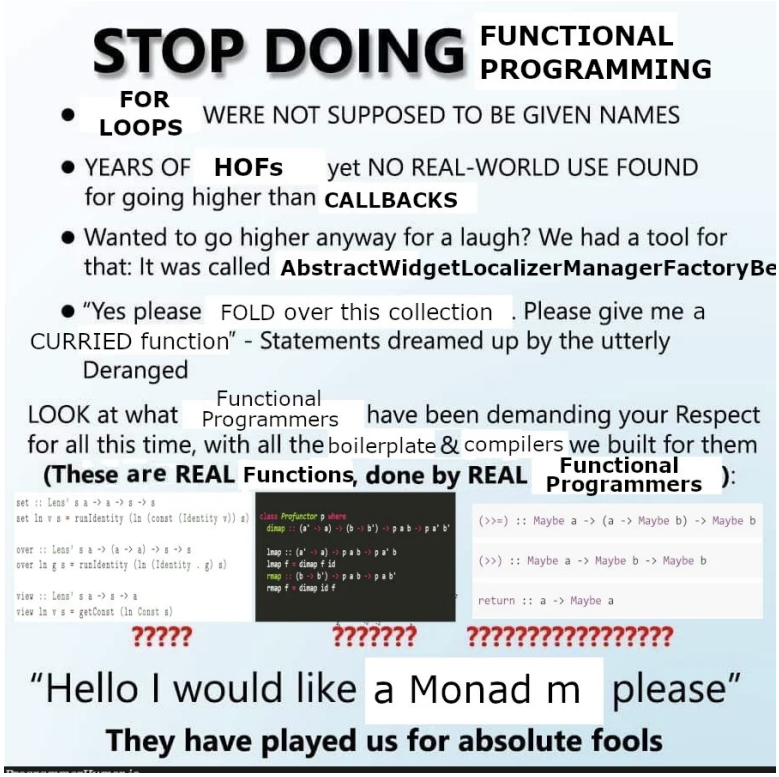
Sapienza Università di Roma

Tecniche di Programmazione Funzionale e Imperativa

libro del corso:
slide del professor Salvo



aglaia norza

28 aprile 2026

thisisaglaia@gmail.com
github.com/AglaiaNorza

Indice

1 Le basi **3**

1.1 Concetti fondamentali 3

1.2 Sintassi, Funzioni e Pattern Matching 4

1.2.1 Definizione e Applicazione 4

1.2.2 Lambda Espressioni e Combinatori 4

1.2.3 Pattern matching 4

1.2.4 Scope locale: `let` e `where` 5

1.3 Sistema dei Tipi e Classi 6

1.3.1 Type Signatures 6

1.3.2 Type Classes 6

1.3.3 Definizione di Tipi: `type` e `data` 7

1.3.4 Istanze e Derivazione (`deriving`) 8

1.3.5 Tipabilità e Let-Polymorphism 8

1.3.6 Semantica Denotazionale: ricorsione e punti fissi 9

1.4 Strutture Dati Predefinite 10

1.4.1 Tipi di Base e Tuple 10

1.4.2 Liste 10

1.5 Funzion(al)i di Ordine Superiore 12

1.5.1 Famiglia dei `fold` (schemi di ricorsione) 12

1.6 Alcuni esempi di stile funzionale 13

2 Lambda Calcolo **15**

2.1 Sintassi e Computazione 15

2.2 Variabili e Combinatori 15

2.2.1 Combinatori noti e strutture di controllo 16
2.3 Ricorsione e Combinatori di Punto Fisso 16

3 Temi avanzati di programmazione funzionale **18**
3.1 Fusion law per foldr 18

1

Le basi

1.1 Concetti fondamentali

- **Valutazione lazy:** Haskell valuta le espressioni solo quando è strettamente necessario (permette di gestire agevolmente strutture dati infinite)
- **Trasparenza referenziale:** non essendoci assegnazioni o memoria mutabile (assenza di side-effects), una variabile identifica sempre lo stesso oggetto (permette di manipolare algebricamente i programmi)
- **Polimorfismo e type inference:** molte funzioni operano su tipi generici (es. `a`, `b`). Il compilatore inferisce automaticamente il *tipo principale* (il più generale possibile); tutti gli altri tipi corretti sono sue istanze.
- **Non-terminazione, Bottom e `undefined`:** in Haskell, ogni tipo possiede implicitamente un valore speciale (chiamato *bottom*, indicato matematicamente con $\perp$) che rappresenta un'eccezione fatale o una computazione che entra in un loop infinito
  - **la costante `undefined`:** è la materializzazione a livello di codice del valore $\perp$ - poiché una computazione che fallisce o non termina può avvenire in qualsiasi contesto (mentre si aspetta un numero, un booleano, ecc.), `undefined` possiede il tipo più generale e polimorfo possibile: `a` (ovvero $\forall\alpha.\alpha$)
  - **Il limite del lambda calcolo (*Occurs check*):** nel lambda calcolo puro, il loop infinito per eccellenza è il combinatore $\Omega = (\lambda x.xx)(\lambda x.xx)$. In Haskell, l'auto-applicazione $\lambda x.xx$ **non è tipabile**. Perché x possa prendere se stesso come argomento, il suo tipo α dovrebbe essere uguale a una funzione che prende α come parametro e restituisce β ($\alpha = \alpha \to \beta$). Questo creerebbe un tipo infinito (durante l'inferenza dei tipi, il compilatore esegue un controllo chiamato *Occurs check* che impedisce alla variabile di tipo α di apparire all'interno della sua stessa definizione, rifiutando di conseguenza l'espressione)
  - **Ricorsione esplicita:** poiché il type system ci vieta di creare loop tramite l'auto-applicazione anonima di $\lambda x.xx$, in Haskell la non-terminazione si esprime unicamente attraverso la ricorsione esplicita. Definendo ad esempio `omega' = omega'`, creiamo intenzionalmente una computazione divergente che il compilatore accetta, assegnandole il tipo più generale `a`.
- **Ideologia funzionale:** l'ideologia funzionale favorisce l'assemblaggio di funzioni semplici tramite composizione (`.`). Si pensa in termini di trasformazioni globali di strutture dati, per poi raffinare in un secondo momento i colli di bottiglia prestazionali.

1.2 Sintassi, Funzioni e Pattern Matching

1.2.1 Definizione e Applicazione

- l'applicazione di funzione: **associa a sinistra** e si scrive senza virgole (`f x y`).
- **aperatori infissi vs prefissi:** le funzioni binarie si usano in modo infisso tra backtick (`10 ‘div‘ 2`), gli operatori infissi in modo prefisso tra parentesi (`(+) 3 4`).

1.2.2 Lambda Espressioni e Combinatori

Le funzioni anonime si scrivono con backslash (`\` simula λ).

```haskell
-- identita' (combinatore I)
i :: t -> t
i x = x           -- definizione standard
i' = \x -> x      -- lambda espressione

-- combinatore K (proiettore del primo argomento) (primo assioma di Hilbert)
k :: t1 -> t2 -> t1
k' = \x y -> x

-- combinatore S (sostituzione) (secondo assioma di Hilbert)
s :: (t2 -> t1 -> t) -> (t2 -> t1) -> t2 -> t
s x y z = x z (y z)
```

1.2.3 Pattern matching

Il pattern matching decompone le strutture dati in base alla loro forma sintattica. L'ordine delle clausole è strettamente sequenziale (viene scelta la prima dall'alto che fa matching).

- **mancanza di unificazione:** non si possono usare variabili identiche nel pattern per verificarne l'uguaglianza (es. `f x x = ...` è invalido)
- **underscore (`_`):** variabile anonima ("don't care") per valori non rilevanti
- **as-pattern (`@`):** permette di dare un nome all'intera struttura mentre la si decompone (es. `xs@(x:txs)`)
- **guardie (`|`):** permettono di definire il comportamento di una funzione valutando sequenzialmente una serie di *predicati booleani* (condizioni logiche); a differenza del pattern matching che controlla la *forma* sintattica dei dati, le guardie valutano *valori e relazioni* (es. `x > 0`, `x == y`)
  - l'ultima guardia è tipicamente `otherwise` (che nel Prelude di Haskell è semplicemente un alias per `True`), che funge da caso di default ("catch-all") per assicurarsi che la funzione restituisca sempre un valore.

```haskell
confronta x y
  | x == y    = "i numeri sono uguali" -- supera la mancanza di unificazione
  | x > y     = "il primo e' maggiore"
  | otherwise = "il secondo e' maggiore"
```

- **equazioni multiple (definizione per casi):** il modo più comune di fare pattern matching è definire la funzione scrivendo più "intestazioni" separate; il compilatore le prova dall'alto verso

il basso: se gli argomenti combaciano con la forma (pattern) della riga, esegue quell'equazione, altrimenti controlla la successiva.

```
myAnd True True = True
myAnd   _    _   = False
```

– è meno comune, ma è possibile fare pattern-matching anche nelle *lambda-espressioni*:

```
myFst' = \(x, y) -> x
```

1.2.4 Scope locale: `let` e `where`

- `let ... in`: costrutto standalone per definire variabili/funzioni locali (utile per strutturare/destrutturare tuple nelle ricorsioni)
- `where`: clausola che qualifica la parte destra di una definizione appena conclusa

```
fibAux n = let (f, fPrec) = fibAux (n-1) in (f + fPrec, f)
-- equivalente a:
fibAux' n = (f + fPrec, f) where (f, fPrec) = fibAux' (n-1)
```

> differenza tra `let` e `where`
>
> una clausola `where` è visibile da tutte le guardie di una specifica equazione. Un `let` definito dopo il simbolo `=` di una guardia è visibile solo all'interno di quella singola guardia. Per condividere un calcolo tra più guardie, il `where` è obbligatorio.

```
-- condivisione tra guardie
analizzaRaggio r
  | area > 100 = "cerchio grande"
  | area < 10  = "cerchio piccolo"
  | otherwise  = "cerchio medio"
  where area = pi * r^2  -- 'area' calcolata una volta e visibile a tutti i pipe

-- inline: (serve il let perche' non c'e' una dichiarazione di funzione)
volumeCilindro r h = (let area = pi * r^2 in area) * h
```

1.3 Sistema dei Tipi e Classi

Haskell è **statically type-safe**: se un programma è tipato correttamente, nessun errore di tipo si verifica durante l'esecuzione.

1.3.1 Type Signatures

In Haskell, grazie al meccanismo di type inference, il compilatore è in grado di dedurre automaticamente il tipo più generale per quasi ogni espressione. Tuttavia, è considerata un'ottima pratica (e in alcuni casi più complessi è strettamente necessario) dichiarare esplicitamente il tipo di una funzione subito sopra la sua definizione.

Questo si fa usando l'operatore `::` (che si legge "ha tipo"). Dichiarare esplicitamente i tipi è utile ai fini della leggibilità del codice e permette di "forzare" il tipo desiderato, aiutando il compilatore a individuare eventuali errori logici in modo molto più preciso.

```haskell
-- dichiarazione esplicita del tipo: prende due Int e restituisce un Int
somma :: Int -> Int -> Int
somma x y = x + y
```

1.3.2 Type Classes

Le classi limitano il polimorfismo imponendo vincoli (o "interfacce") sui tipi. Più che semplici insiemi, rappresentano classificazioni di tipo algebrico: raggruppano i tipi su cui sono definite certe operazioni. Le classi possono formare gerarchie, dove una sottoclasse "estende" la superclasse ereditandone le operazioni e aggiungendone di nuove.

- **Eq (Equality Types):** comprende i tipi che ammettono l'uguaglianza (`==`) e la sua negazione (`/=`). Tutti i tipi base (Bool, Int, Float, ecc.) sono istanze di Eq, così come le liste e tuple costruite su tipi Eq

  definizione interna di Eq

  ```haskell
  class Eq a where
    (==), (/=) :: a -> a -> Bool
    -- le classi possono contenere codice (metodi di default)
    x /= y = not (x == y)
  ```

- **Ord (Ordered Types):** sottoclasse di `Eq` - tipi ordinabili che supportano operazioni come `<=`, `<`, `>=`, `>`, `min` e `max`.
  - per definire un'istanza di Ord, è sufficiente fornire il codice per l'operatore `<`, il resto può essere interdefinito.

- **Num:** la classe più generica per i tipi numerici. È sottoclasse di `Eq` e `Show`. Implementa operazioni aritmetiche base (`+`, `-`, `*`), oltre a valore assoluto (`abs`) e segno (`signum`).

  i numeri interi costanti sono polimorfi rispetto alla classe `Num`:

  ```haskell
  > :t 3
  3 :: Num a => a  -- 3 puo' essere un Int, Integer, Float, ma non un Char
  ```

- **Altre classi utili:**

- **Show:** tipi che possono essere stampati (convertiti in `String`) tramite il metodo `show`. Se provate a stampare a video il risultato di una funzione che non appartiene a `Show` (es. una lambda anonima come `\x -> x`), l'interprete darà errore.
- **Read:** permette di trasformare una `String` in un valore tramite il metodo `read`
- **Integral:** sottoclasse di `Num` per i numeri interi (es. Int, Integer); aggiunge le operazioni di divisione intera `div` e modulo `mod`
- **Fractional:** sottoclasse di `Num` per i numeri frazionari (es. Float, Rational); aggiunge l'operatore di divisione reale `(/)` e `recip`

1.3.2.1 Vincoli di classe

I vincoli di classe si indicano prima del tipo vero e proprio, separati dalla doppia freccia `=>`. Vincoli multipli si racchiudono tra parentesi e indicano che la funzione opera su un tipo che deve soddisfare tutte le classi elencate.

```haskell
-- esempio: mcd richiede che il tipo 'a' supporti sia l'ordine (<, ==)
-- sia le operazioni aritmetiche (-)
mcd :: (Ord a, Num a) => a -> a -> a
mcd x y
  | x == y    = x
  | x < y     = mcd (y - x) x
  | otherwise = mcd (x - y) y
```

1.3.3 Definizione di Tipi: `type` e `data`

- **`type` (sinonimi):** danno nuovi nomi a tipi esistenti (es. `type Casella = (Int, Int)`)
- **`data` (tipi algebrici):** costruiscono nuovi tipi tramite costruttori finiti, parametrici o ricorsivi. Possono essere dichiarati istanze di classi.

```haskell
data SetteNani = Eolo | Pisolo | Brontolo          -- finiti
data Either a b = Left a | Right b                 -- unioni disgiunte
data Maybe a = Just a | Nothing                    -- gestione fallimenti
data List a = Nil | Cons a (List a)                -- ricorsivi/induttivi
data BTree a = Foglia a | Radice (BTree a) (BTree a)
```

1.3.3.1 Il costruttore di tipo `Maybe`

Il tipo `Maybe` è il tipo di computazioni che possono fallire. In Haskell, è il tipo con cui si rappresentano comunemente le eccezioni in maniera sicura e controllata dal sistema di tipi (corrisponde a `optional`)

Un valore di tipo `Maybe` può assumere due forme:

- `Just v`: contiene un valore `v` (la computazione ha avuto successo e restituisce qualcosa)
- `Nothing`: non contiene nulla (rappresenta una computazione indefinita o fallita)

Si può utilizzare per le *funzioni parziali* (che non sono definite per tutti i possibili valori di input (eg divisione per zero)). Tramite il pattern matching, possiamo "spacchettare" il dato per estrarne il contenuto.

```haskell
-- definizione del tipo predefinito
data Maybe a = Just a | Nothing

-- 1. costruzione di un Maybe: divisione sicura
-- Se il divisore e' 0 restituisce Nothing, altrimenti restituisce Just il
    risultato
safediv :: Integral a => a -> a -> Maybe a
safediv n 0 = Nothing
safediv n m = Just (n `div` m)

-- 2. destrutturare un Maybe: funzione "inversa" per estrarre il valore
extract :: Maybe a -> a
extract (Just n) = n
extract Nothing  = undefined

-- extract (safediv 30 0) -> *** Exception: Prelude.undefined
```

1.3.4 Istanze e Derivazione (`deriving`)

La keyword `deriving` è un meccanismo automatico che permette al compilatore di generare le istanze per le classi predefinite senza dover scrivere codice manuale.

- **Eq:** genera un'uguaglianza basata sulla sintassi
- **Ord:** deriva l'ordine lessicografico in base alla sequenza dei costruttori
- **Show/Read:** la conversione usa i nomi dei costruttori come stringhe
- **Enum:** permette di generare liste per enumerazione (es. `[Lun..Ven]`)

```haskell
-- istanze automatiche tramite deriving
data Giorni = Lun | Mar | Mer | Gio | Ven | Sab | Dom
    deriving (Eq, Ord, Show, Enum, Read)

-- esempio di istanza manuale
instance Eq Bool where
    False == False = True
    True == True   = True
    _ == _         = False
```

1.3.5 Tipabilità e Let-Polymorphism

- Non tutto è tipabile in Haskell. Il classico esempio è l'auto-applicazione pura:
  - `omega = \x -> x x` non è tipabile perché richiederebbe un tipo infinito ($t = t \to t_1$), portando a una teoria dei tipi non decidibile (e quindi a tipi non inferibili dal compilatore)
- Dal punto di vista del calcolo, `let x = N in M` è equivalente all'applicazione di una lambda: `(\x -> M) N`. Tuttavia, il compilatore Haskell tratta i due casi in modo diverso durante l'inferenza dei tipi.
- **Let-Polymorphism:**

  Il costrutto `let` permette di assegnare a una variabile un tipo polimorfo che viene "generalizzato" prima di essere usato nel corpo (`in`).
  - **tipabile:** `let x = \y -> y in x x` è tipabile.

Haskell vede che `x` è l'identità ($a \to a$) e, nel corpo `x x`, può istanziare il primo `x` come $(a \to a) \to (a \to a)$ e il secondo come $(a \to a)$.

– **non tipabile:** `(\x -> x x) (\y -> y)` **non** è tipabile.
Qui il compilatore deve tipare la funzione `\x -> x x` prima di sapere che `x` sarà l'identità. Come visto sopra, `\x -> x x` è intrinsecamente non tipabile.

> Lemma 1: **Terminazione**
>
> Tutti i lambda-termini tipabili nel sistema dei tipi di Haskell (che non usino ricorsione esplicita tramite equazioni) **terminano**.

1.3.6 Semantica Denotazionale: ricorsione e punti fissi

Haskell è un linguaggio dichiarativo: una definizione non è un comando, ma un'equazione. Quando definiamo una funzione ricorsiva f, stiamo scrivendo un'equazione del tipo $f = \mathcal{E}(f)$, dove $\mathcal{E}$ è un'espressione che contiene f stessa.

- **Bottom ($\perp$)** rappresenta il "non-valore". Indica un calcolo che non termina (loop infinito) o che fallisce (errore). In Haskell, ogni tipo t contiene implicitamente $\perp$.
- Se la definizione è un'equazione, la "semantica" (ovvero la funzione reale prodotta dal compilatore) deve essere una soluzione dell'equazione. Matematicamente, una soluzione di $f = T(f)$ è un **punto fisso** del funzionale T.

> Approfondimento: ricorsione e non-terminazione: `omega'`
>
> Prendiamo la funzione
>
> ```haskell
> omega' x = omega' x
> -- > :t omega'
> -- omega' :: t -> t1
> ```
>
> Questa funzione ha il tipo più generico possibile (`t -> t1`) perché, non terminando mai, non produce mai un valore reale.
>
> Rimuovendo x, otteniamo $omega' = I(omega')$, dove I è l'identità.
>
> Qualsiasi funzione f soddisfa $f = I(f)$. Quale scegliere?
>
> - Haskell sceglie la funzione che "fa il minimo indispensabile" per soddisfare l'equazione. Si usa l'ordinamento $\sqsubseteq$ dove $f \sqsubseteq g$ significa che g è "più definita" di f.
> - **Costruzione di Kleene (least-fixpoint):**
> il computer "costruisce" la funzione partendo dal nulla ($\perp$):
>
> $$f_0 = \perp \quad \text{(non so nulla)}$$
>
> $$f_1 = T(f_0) \quad \text{(applico la definizione una volta)}$$
>
> $$f_\infty = \text{limite della catena} = \mathit{lfp}(T)$$
>
> Nel caso di `omega'`, la catena è $\{\perp, \perp, \dots\}$. Il limite è $\perp$. La semantica di `omega'` è dunque la funzione che restituisce sempre "non-valore".

Il punto fisso è quindi la "semantica" perché è l'unico modo per dare un valore univoco e calcolabile a una definizione circolare. Il *minimo* punto fisso garantisce che il significato sia quello "prodotto" effettivamente dal calcolo ricorsivo.

`myundefined`

Mentre `omega'` è una *funzione* che non termina, possiamo definire un *valore* che non termina:

```
myUndefined = myUndefined
-- > :t myUndefined
-- myUndefined :: a
```

- Poiché `myUndefined` ha tipo `a`, esso **appartiene a tutti i tipi**. È l'implementazione "home-made" della costante `undefined` di Haskell. Rappresenta il valore $\perp$ per ogni possibile tipo del linguaggio.
- Matematicamente, `myUndefined` è il **punto fisso dell'identità**:
  (1) la definizione `x = x` corrisponde all'equazione funzionale $x = I(x)$
  (2) la soluzione di questa equazione è, per definizione, il punto fisso dell'identità I
  (3) poiché Haskell usa la semantica del minimo punto fisso, e l'identità applicata al valore nullo ($\perp$) restituisce ancora $\perp$, il risultato è la non-terminazione
  (4) questo spiega perché `myUndefined` ha tipo `a`: non essendo "sceso" verso nessun valore specifico (come un intero o una stringa), rimane una pura astrazione di errore/loop che può "fingere" di essere qualsiasi tipo.

Haskell permette quindi di manipolare la non-terminazione come un valore reale. Ogni tipo in Haskell è in realtà "puntato" (*pointed*), ovvero contiene sempre almeno un valore: $\perp$.

1.4 Strutture Dati Predefinite

1.4.1 Tipi di Base e Tuple

- **Booleani:** `Bool` (`True`, `False`). Operatori: `not`, `&&`, `||`.
- **Interi:** `Int` (dimensione fissa), `Integer` (precisione illimitata).
- **Tuple:** n-uple fisse e disomogenee (es. `(String, Bool)`)
  offrono le funzioni *su coppie*: `fst`, `snd` (prendono primo e secondo argomento)

1.4.2 Liste

Sequenze di elementi omogenei. Definite dal costruttore vuoto `[]` e dall'operatore infisso *cons* `(:)`. Le stringhe sono `[Char]`.

Zucchero sintattico: `[1,2,3]` equivale a `1:2:3:[]`.

```
-- enumerazioni
[m..n]          -- range finito (lista con elementi da m a n)
[m, n..p]       -- range con step (es. [1, 3..11] dispari)
```

```
[m..]            -- lista infinita (lazy)
```

Haskell supporta le **list comprehension** (definizioni set-like del contenuto di una lista)

```
-- list comprehension (spesso tradotte internamente in map/filter)
[x^2 | x <- [1..5]]                -- quadrati
[x | x <- xs, x `mod` 2 == 0]      -- filtri / guardie
[(x,y) | x <- xs, y <- ys]         -- prodotto cartesiano
```

1.4.2.1 Funzioni su Liste

- **Estrazione:** `head`, `tail`. `last`, `init`. `(!!)` per estrarre all'indice n

```
head [1, 2, 3]  -- risultato: 1
tail [1, 2, 3]  -- risultato: [2, 3]
last [1, 2, 3]  -- risultato: 3
init [1, 2, 3]  -- risultato: [1, 2]
[1, 2, 3] !! 1  -- risultato: 2
```

- **Ispezione:** `null`, `length`

```
null []          -- risultato: True
length [1, 2, 3] -- risultato: 3
```

- **Taglio:** `take n`, `drop n`, `splitAt n` (tupla con take e drop).

```
take 2 [1, 2, 3, 4]    -- risultato: [1, 2]
drop 2 [1, 2, 3, 4]    -- risultato: [3, 4]
splitAt 2 [1, 2, 3, 4] -- risultato: ([1, 2], [3, 4])
```

- **Generali:** `++` (concatenazione), `reverse`

```
[1, 2] ++ [3, 4]  -- risultato: [1, 2, 3, 4]
reverse [1, 2, 3] -- risultato: [3, 2, 1]
```

- **Matematica/logica:** `sum`, `minimum`, `maximum`. `and`, `or`. `any p`, `all p` (predicati su almeno uno o tutti)

```
sum [1, 2, 3]        -- risultato: 6
minimum [5, 2, 9]    -- risultato: 2
maximum [5, 2, 9]    -- risultato: 9
and [True, False]    -- risultato: False
or [True, False]     -- risultato: True
any (> 2) [1, 2, 3] -- risultato: True (almeno uno è > 2)
all (> 0) [1, 2, 3] -- risultato: True (tutti sono > 0)
```

- **Liste annidate/multiple:** `concat` (appiattisce liste di liste), `zip` (unisce liste in coppie, fermandosi alla più corta)

```
concat [[1, 2], [3, 4]]    -- risultato: [1, 2, 3, 4]
zip [1, 2] ['a', 'b', 'c'] -- risultato: [(1, 'a'), (2, 'b')]
```

1.5 Funzion(al)i di Ordine Superiore

I funzionali sono funzioni che prendono altre funzioni come argomento o le restituiscono come risultato. Favoriscono la **composizionalità**, permettendo di costruire programmi complessi tramite piccoli blocchi riutilizzabili.

- **curryficazione:** si fonda sull'isomorfismo $A \times B \to C \cong A \to (B \to C)$. Le versioni curryficate permettono di passare un solo argomento alla volta, favorendo l'applicazione parziale.
- **η-regola:** è possibile omettere un parametro in una definizione se esso compare identico alla fine di entrambi i lati dell'equazione.

```
-- composizione (.) : Equivale a f(g(x))
(.) f g x = f (g x)

-- curry e uncurry (Isomorfismo A x B -> C  < = = >  A -> (B -> C))
curry :: ((a, b) -> c) -> a -> b -> c
uncurry :: (a -> b -> c) -> (a, b) -> c

-- flip: inverte l'ordine dei parametri
flip :: (a -> b -> c) -> b -> a -> c
myFlip f a b = f b a

-- flip (:) [2, 3] 1
-- [1, 2, 3]

-- map e filter
map :: (a -> b) -> [a] -> [b]          -- proprieta': map (f . g) = map f . map g
filter :: (a -> Bool) -> [a] -> [a]

-- zipWith: applica f. binaria agli elementi di due liste
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

1.5.1 Famiglia dei `fold` (schemi di ricorsione)

I `fold` astraggono il concetto di ricorsione strutturale sulle liste, sostituendo i costruttori della struttura dati (l'operatore `(:)` e la lista vuota `[]`) con funzioni e valori specifici. Se l'operatore `#` è associativo e ha `e` come elemento neutro, allora `foldl (#) e` = `foldr (#) e`.

- **`foldr` (right fold):** associa a destra; la riduzione avviene "al ritorno" dalle chiamate ricorsive (costruisce espressioni).
  - sostituisce ricorsivamente il costruttore `(:)` con la funzione `f` data e `[]` con il valore base `z`.
  - espansione: `foldr f z [x1, x2, x3]` diventa `x1 ‘f‘ (x2 ‘f‘ (x3 ‘f‘ z))`
  - può terminare anche su liste infinite se la funzione `f` non è stretta nel suo secondo argomento (ovvero se non ha sempre bisogno di valutare la ricorsione per restituire un risultato).

```haskell
-- somma con foldr
foldr (+) 0 [1, 2, 3]
-- valutazione: 1 + (2 + (3 + 0)) = 6

-- implementazione di map usando foldr
myMap f xs = foldr (\x acc -> f x : acc) [] xs
```

- **`foldl` (left fold):** associa a sinistra; simula un ciclo iterativo passando un accumulatore "in avanti"
  - valuta l'espressione partendo da sinistra e procedendo verso l'interno - valuta tutto solo alla fine della lista
  - espansione: `foldl f z [x1, x2, x3]` diventa `((z ‘f‘ x1) ‘f‘ x2) ‘f‘ x3`
  - *nota bene:* In `foldl`, la funzione prende come primo argomento l'accumulatore, e come secondo l'elemento corrente della lista.

```haskell
-- sottrazione con foldl
foldl (-) 0 [1, 2, 3]
-- valutazione: ((0 - 1) - 2) - 3 = -6

-- sottrazione con foldr per confronto
foldr (-) 0 [1, 2, 3]
-- valutazione: 1 - (2 - (3 - 0)) = 2

-- reverse usando foldl
myReverse xs = foldl (\acc x -> x : acc) [] xs
```

- **`foldr1` e `foldl1`:** varianti che non richiedono un valore iniziale esplicito, poiché usano il primo o l'ultimo elemento come base; *danno errore su lista vuota*.

```haskell
-- utile quando non esiste un elemento neutro sensato (es. min/max assoluto)
myMinimum = foldr1 min
myMinimum [5, 3, 8] -- risultato: 3
-- myMinimum []     -- genera errore a runtime
```

1.6 Alcuni esempi di stile funzionale

Lo stile funzionale evita accumulatori ed indici, preferendo la manipolazione globale delle strutture dati.

- **prodotto scalare:** può essere implementato in diversi modi (flessibilità dei funzionali):

```haskell
-- accoppia gli elementi, trasforma "*" in una funzione che accetta una tupla
-- applica la moltiplicazione agli elementi, e somma i prodotti
ps1 xs ys = sum (map (uncurry (*)) (zip xs ys))

-- crea una lista di funzioni parzialmente applicate
-- (da [5, 4] si ottiene [(5*), (4*)])
-- zApp/applyL prende la lista di funzioni ^ e la applica sulla seconda lista
ps2 xs ys = sum (applyL (map (*) xs) ys)

-- zipWith fa zip e map in un passo solo
```

```
ps3 xs ys = sum (zipWith (*) xs ys)
```

- controllare se una lista è ordinata:

```
ordinata xs = and (zipWith (<=) xs (tail xs))
```

- **ricerca su lista** (`findVPH`): si può implementare accoppiando la lista agli indici (`zip [0..] xs`) e filtrando per il valore cercato.

2

Lambda Calcolo

Ideato da Alonzo Church nel 1931, il λ-calcolo nasce per esplicitare il *processo meccanico di calcolo* (sintassi) di una funzione, in contrasto con la visione matematica classica che vede la funzione solo come una relazione statica tra insiemi (semantica).

2.1 Sintassi e Computazione

La sintassi del λ-calcolo è minimalista ed è composta da tre regole:

- **variabile:** $x, y, z \dots$
- **astrazione:** $\lambda x.M$ (corrisponde alla definizione di una funzione con parametro x e corpo M)
- **applicazione:** (MN) (corrisponde all'applicazione della funzione M all'argomento N)

in BNF, quindi, la grammatica del lambda-calcolo è quindi:

$$T ::= V \mid \lambda V.T \mid (T\ T)$$

> Regole di associazione
>
> - **l'applicazione associa a sinistra:** $FN_1N_2 \dots N_n$ equivale a $(\dots(FN_1)\dots N_n)$
> - **l'astrazione associa a destra:** $\lambda x_1 \dots x_n.M$ equivale a $\lambda x_1.(\lambda x_2.(\dots(\lambda x_n.M)\dots))$

La computazione avviene tramite la β**-riduzione** (beta-regola): un termine della forma $(\lambda x.M)N$ si dice *redex* e si riduce sostituendo tutte le occorrenze di x in M con il termine N. Formalmente:

$$(\lambda x.M)N \to_\beta M[N/x]$$

Un termine che non contiene più β-redex è detto in **forma normale** e rappresenta un valore finale (il risultato della computazione).

2.2 Variabili e Combinatori

- **Variabili libere (FV) e legate:** in $\lambda x.M$, la variabile x è *legata* (bound) all'astrazione. Qualsiasi variabile non legata è *libera* (free).
- α**-conversione (alfa-regola):** per evitare la cattura accidentale di variabili libere durante la sostituzione, si possono rinominare le variabili legate (es. $\lambda x.x \equiv \lambda y.y$).

  - è buona pratica seguire la "convenzione di Barendregt", che stabilisce che, all'interno di un termine lambda o di un insieme di termini, tutte le variabili legate devono avere nomi distinti tra loro e diversi da tutte le variabili libere presenti nel termine stesso

- I termini tali che ($FV(M) = \emptyset$) (λ-termini chiusi), ovvero senza variabili libere, sono chiamati **combinatori**

2.2.1 Combinatori noti e strutture di controllo

Il λ-calcolo puro non ha tipi di dato predefiniti: tutto (booleani, numeri, if-then-else) deve essere codificato tramite funzioni.

- **Identità (I):** $I \equiv \lambda x.x$
- **Cancellatori (booleani):**

  True (K): $K \equiv \lambda xy.x$ (prende due argomenti e restituisce il primo)

  False (O): $O \equiv \lambda xy.y$ (prende due argomenti e restituisce il secondo)
- **If-Then-Else:** grazie alla definizione dei booleani, la struttura condizionale è codificata come un'applicazione: $if\ x\ then\ M\ else\ N \equiv xMN$. Se x è True (K) selezionerà M, se è False (O) selezionerà N.
- **Compositori (S):** $S \equiv \lambda xyz.xz(yz)$ (si può dimostrare che $SKK \approx I$).
- **Duplicatori e divergenza:**

  $\omega \equiv \lambda x.xx$ (auto-applicazione)

  $\Omega \equiv \omega\omega \to \omega\omega \to \dots$ rappresenta la **funzione indefinita**, il loop infinito (non tipabile)

2.2.1.1 Numeri di Church e iteratori

I **Numerali di Church** rappresentano i numeri naturali come funzioni di ordine superiore che *iterano* un'operazione. Il numero n è una funzione che prende una funzione (es. successore) s e un valore di partenza (es. zero) z, e applica s ad z esattamente n volte

$$\underline{n} \equiv \lambda sz.s(s(\dots(sz)\dots)) \quad [n \text{ applicazioni}]$$

- $\underline{0} \equiv \lambda sz.z$ (equivale a `False`)
- $\underline{3} \equiv \lambda sz.s(s(s(z)))$

Per calcolare il successore ($+1$) di un numero di Church, basta creare una funzione che applica s una volta in più: $succ \equiv \lambda xsz.s(xsz)$ (o anche $\lambda xsz.xs(sz)$)

I numerali di Church non sono quindi solo dati, ma **iteratori intrinseci**.

2.3 Ricorsione e Combinatori di Punto Fisso

Per definire funzioni ricorsive complesse il λ-calcolo necessita di un principio di ricorsione generale.

> Thm. 1: **Combinatore di Punto Fisso**
>
> Nel λ-calcolo esiste un termine Y (detto **Combinatore di Punto Fisso**) tale che, per ogni termine M, si ha $YM \to M(YM)$.

Il combinatore permette di trovare quindi il punto fisso di una funzione (quel valore x tale che $f(x) = x$)

Due combinatori di punto fisso sono:

- **Combinatore di Turing (Y_T):** definito come $Y_T = \theta\theta$ dove $\theta \equiv \lambda xy.y(xxy)$.

$$\begin{aligned} Y_T M &\equiv (\theta\theta)M \\ &\equiv ((\lambda xy.y(xxy))\theta)M \\ &\to_\beta (\lambda y.y(\theta\theta y))M \\ &\to_\beta M(\theta\theta M) \\ &\equiv M(Y_T M) \end{aligned}$$

- **Combinatore di Curry (Y_C):** definito come $Y_C \equiv \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$. Se applicato all'identità I, questo combinatore riduce a Ω (divergenza).

$$\begin{aligned} Y_C M &= (\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))M \\ &\to_\beta (\lambda x.M(xx))(\lambda x.M(xx)) \\ &\to_\beta M((\lambda x.M(xx))(\lambda x.M(xx))) \\ &\equiv M(Y_C M) \end{aligned}$$

(l'ultima equivalenza $\equiv$ vale perché $(\lambda x.M(xx))(\lambda x.M(xx))$ è la forma ridotta di $Y_C M$)

Thm. 2: **Tesi di Church-Turing**

Ogni funzione calcolabile è λ-definibile.

3

Temi avanzati di programmazione funzionale

3.1 Fusion law per foldr

La **Fusion Law** per `foldr` è un principio di induzione generalizzato che permette di trasformare la composizione di una funzione con una `foldr` in un'unica fold. Essa stabilisce le condizioni per cui vale l'uguaglianza:

`f . foldr g a = foldr h b`

L'obiettivo è quindi trovare funzioni o valori `h` e `b` tali che l'applicazione di `f` al risultato di una `foldr` sia equivalente ad una nuova `foldr` che lavora direttamente su di essi - l'utilità computazionale risiede nella capacità di "fondere" due passaggi (una trasformazione `f` applicata dopo un fold con `g`) in un unico ciclo sulla struttura dati, evitando allocazioni intermedie.

Thm. 3: **Fusion law**

Siano `f`, `g`, `h` funzioni e `a`, `b` valori. L'uguaglianza `f . foldr g a = foldr h b` è garantita se sono soddisfatti i seguenti tre requisiti:

(1) `f` è stretta (`f undefined = undefined`) (così che valga anche nel caso di liste parziali/indefinite)

(2) **caso base**: `f a = b` - `f` applicata all'accumulatore iniziale della prima `foldr (a)` deve dare come risultato l'accumulatore iniziale della seconda (`b`)

(3) **passo induttivo** (condizione di fusione): per ogni `x`, `y`, deve valere:

`f (g x y) = h x (f y)` - in questo modo, l'operazione `h` del nuovo `foldr` riproduce esattamente l'effetto di `g` seguito da `f`

parallelo con un omorfismo

La fusion law può essere interpretata come la dimostrazione che `f` agisce come un **omomorfismo** tra due strutture algebriche di calcolo:

- **preservazione dell'identità**: `f a = b` mappa l'elemento neutro (accumulatore iniziale) del primo dominio in quello del secondo, analogamente a $f(1_A) = 1_B$.
- **preservazione dell'operazione**: La condizione `f(g x y) = h x (f y)` ricalca la proprietà omomorfica $f(a \bullet b) = f(a) \circ f(b)$.

- *sorgente* `( g x y )`: rappresenta l'operazione di combinazione di un elemento `x` con il risultato parziale `y` tramite la funzione `g`
- *destinazione* `( h x (f y))`: rappresenta la nuova operazione `h` che combina `x` con il risultato già trasformato da `f`

- **NB**: in `h x (f y)`, l'elemento `x` non viene trasformato da `f` - nella struttura della lista, `x` è un dato "guida" di tipo A, mentre `y` è il risultato della ricorsione (accumulatore) di tipo B. La fusion law mira a trasformare il *risultato complessivo* del fold, non i singoli elementi contenuti nella lista.

Derivazione formale

Per dimostrare la validità dell'uguaglianza `f . foldr g a = foldr h b`, analizziamo separatamente i due lati della relazione applicando i principi di induzione sulla struttura della lista:

Ricordando la definizione di `foldr` basata sulla decomposizione della lista:

- `foldr g a [] = a`
- `foldr g a (x:xs) = g x (foldr g a xs)`

Caso `[]` (base):

- **sinistra**: `(f . foldr g a) []`
  `= f (foldr g a [])` *(def di .)*
  `= f a` *(def di foldr)*
- **destra**: `foldr h b []`
  `= b` *(def di foldr)*
- da cui deriva la prima condizione: `f a = b`

Caso `(x:xs)` (induttivo):

- **sinistra**: `(f . foldr g a) (x:xs)`
  `= f (foldr g a (x:xs))` *(def di .)*
  `= f (g x (foldr g a xs))` *(def di foldr)*
- **destra**: `foldr h b (x:xs)`
  `= h x (foldr h b xs)` *(def di foldr)*
  `= h x (f (foldr g a xs))` *(induzione)*
- ponendo `y = foldr g a xs`, deriviamo la condizione di fusione:

  `f (g x y) = h x (f y)`